ВОПРОСЫ КОМИССИИ

Каверзные вопросы по ВКР и подробные защитные ответы

Проект: EH/Unwind Analyzer - анализ x64 PE exception/unwind metadata

Что это за документ	Как использовать
Количество вопросов	59 вопросов, сгруппированных по смыслу: цель, PE/unwind, признаки, архитектура, тестирование, ограничения и провокации.
Формат ответа	У каждого вопроса есть короткая позиция, подробное объяснение и фраза, которой можно закрыть давление комиссии.
Главная стратегия	Не обещать невозможного. Говорить, что проект является объяснимым анализатором EH/unwind metadata, а не универсальным антивирусом.

Дата подготовки: 14.07.2026 20:17

Как защищаться в целом

Главная ошибка на защите - пытаться представить проект как универсальный антивирус или абсолютный детектор вредоносности. Правильная позиция другая: это специализированный, объяснимый и проверяемый анализатор x64 PE exception/unwind metadata. Он работает с конкретными структурами: PE headers, section table, exception directory, RUNTIME_FUNCTION, UNWIND_INFO, handler flags, HandlerRVA, section permissions и runtime exception evidence.

Пять опорных тезисов

- Проект не заменяет антивирус, sandbox или полный reverse engineering. Он закрывает отдельный сложный слой анализа.

- Все 27 findings имеют технический смысл и привязаны к конкретным структурам или корреляциям.

- Score - triage aid, а не вероятность вредоносности. Confidence - инженерная уверенность, а не статистическая вероятность.

- Ограничения честно названы: PE32+ AMD64, static metadata checks, dynamic Windows-only, no full .text disassembly.

- Качество подтверждается не одним примером, а тестами, regression samples, schema validation, mutations, sanitizers, fuzzing и Windows smoke scenarios.

Самая короткая формулировка проекта

Моя ВКР - это defensive binary analysis tool, который разбирает x64 PE exception/unwind metadata, выявляет структурные и эвристические аномалии, формирует объяснимый отчет и помогает аналитику вручную проверить подозрительные места.

Карта опасных зон

Зона атаки комиссии	Что они могут пытаться доказать	Как отбиваться
Нужность	Такое уже есть в Windows/AV/EDR.	Разделить detection/blocking и explainable EH/unwind analysis.
Новизна	PE давно известен.	Показать связку parser + evidence matrix + reports + correlation.
Пороги	Цифры произвольные.	Объяснить engineering calibration и отсутствие претензии на вероятность.
False positives	Нормальный файл тоже может сработать.	Подчеркнуть findings, confidence, mitigations, manual review.
Ограничения	Не поддерживает все форматы.	Объяснить фокус на x64 Windows unwind как осознанную границу.
Язык C	Можно было проще на Python.	Объяснить низкоуровневость, bytes, Windows API, standalone CLI.
Тестирование	Как доказать, что работает?	Перечислить unit/regression/schema/semantic/mutations/sanitizers/fuzz/dynamic smoke.

1. Смысл проекта, новизна и целевая аудитория

1. Зачем вообще нужен ваш проект, если Windows, антивирусы и EDR уже анализируют PE-файлы?

Короткая позиция:

Мой проект решает не задачу блокировки угроз, а задачу объяснимого анализа конкретного слоя PE: x64 exception/unwind metadata.

Подробный ответ:

Антивирус или EDR обычно отвечает на вопрос: опасен ли файл и нужно ли его заблокировать. Мой инструмент отвечает на другой вопрос: что именно в .pdata, RUNTIME_FUNCTION, UNWIND_INFO, handler flags, HandlerRVA и размещении metadata выглядит несогласованным или подозрительным. Это не замена защитному продукту, а диагностический инструмент для reverse engineering и исследования. Его ценность в том, что он показывает структурные причины, affected count, representative RVA, семейство evidence, score, confidence и шаги ручной проверки. То есть пользователь видит не черный ящик, а объяснимую карту проблем.

Если комиссия давит:

> **Если меня давят сравнением с антивирусом, я отвечаю: антивирус принимает эксплуатационное решение, а мой проект строит объяснимый отчет по узкой низкоуровневой области, которую удобно проверять руками.**

2. В чем научная или инженерная новизна, если PE-формат давно известен?

Короткая позиция:

Новизна не в изобретении PE-формата, а в связке parsing + heuristic evidence matrix + explainable reporting + optional runtime correlation для EH/unwind-слоя.

Подробный ответ:

PE действительно известен, но известность формата не означает наличие готовой объяснимой матрицы признаков именно для exception/unwind metadata. В проекте выделены 27 findings, разделенные на evidence families: table_integrity, unwind_integrity, metadata_placement, unwind_reuse, handler_dominance. Каждый finding имеет severity, вес, affected count и смысл. Дополнительно проект отделяет suspicion score от confidence, применяет mitigations, не считает strict ABI warnings подозрительностью по умолчанию, формирует JSON/HTML/CSV/Ghidra-CSV и умеет сопоставлять часть статических сигналов с runtime-событиями.

Если комиссия давит:

> **Формулировка для защиты: я не утверждаю, что открыл PE. Я сделал объяснимую инженерную систему анализа узкого, сложного и практически полезного механизма.**

3. Кто реальная целевая аудитория, если студентам сложно, а крупные компании имеют свои внутренние инструменты?

Короткая позиция:

Аудитория - reverse engineers, malware analysts, преподаватели, студенты старших курсов, исследователи PE, небольшие команды без закрытых внутренних платформ.

Подробный ответ:

Да, у крупных компаний могут быть внутренние инструменты. Но это не отменяет ценность открытого, воспроизводимого и объяснимого инструмента для обучения, исследования и независимого анализа. Студентам он полезен не как простая игрушка, а как мост от теории PE к реальным структурам Windows x64. Небольшим командам и независимым аналитикам он полезен тем, что быстро готовит отчет по unwind metadata и дает точки ручной проверки. Преподавателю он полезен как демонстрация низкоуровневой архитектуры, тестирования и документированной эвристической модели.

Если комиссия давит:

> **Я не позиционирую проект как массовый пользовательский продукт. Это специализированный инструмент для тех, кто анализирует бинарники и хочет объяснимый отчет по EH/unwind.**

4. Почему крупные компании не выкладывают такие инструменты в открытый доступ?

Короткая позиция:

Потому что внутренние инструменты часто содержат закрытые правила, пороги, телеметрию, тестовые корпуса и данные о методах атак.

Подробный ответ:

У крупных компаний может быть похожая функциональность внутри EDR, sandbox, reverse platform или threat intelligence workflow. Но они редко публикуют все правила, потому что в них есть внутренние scoring-модели, thresholds, correlation rules, проверочные корпуса, сведения об обходах и operational knowledge. Публикация такого набора может помочь злоумышленнику адаптировать samples. Мой проект не раскрывает коммерческие секреты и не претендует на корпоративную телеметрию. Его ценность в открытости, воспроизводимости и учебно-исследовательской направленности.

Если комиссия давит:

> **Если комиссия говорит 'у корпораций это уже есть', ответ: возможно, но закрытость корпоративных решений и открытая объяснимая ВКР - это разные категории результата.**

5. Если проект не доказывает вредоносность, зачем тогда score?

Короткая позиция:

Score нужен для triage, то есть для приоритизации ручной проверки, а не для юридического или антивирусного доказательства.

Подробный ответ:

В README и evidence matrix прямо указано: score - transparent triage aid, not statistical probability. Он помогает быстро понять, насколько много и насколько сильных сигналов найдено. Например, один weak finding не должен звучать так же, как несколько high findings из разных семейств. Поэтому score работает вместе с severity, confidence, mitigations и manual validation steps. Это делает отчет удобным для аналитика: сначала он видит общую картину, потом идет к конкретным RVA и структурам.

Если комиссия давит:

> **Я не говорю 'score 70 значит 70% вирус'. Я говорю 'score 70 значит набор статических evidence достаточно сильный для приоритетной ручной проверки'.**

6. Может ли вредоносный файл пройти ваш анализ без findings?

Короткая позиция:

Да, может. Проект не является полным детектором вредоносности и честно ограничен EH/unwind-областью.

Подробный ответ:

Если malware не использует необычную exception/unwind metadata, а ведет себя вредоносно через сеть, imports, скриптовую логику, credential theft или injection, проект может не найти подозрительных unwind-признаков. Это не провал модели, а следствие четкой области применения. Любой узкоспециализированный анализатор имеет границы. Правильная интерпретация: отсутствие findings означает, что в проверяемом EH/unwind-слое не найдено значимых аномалий по текущей матрице, а не что файл безопасен во всех отношениях.

Если комиссия давит:

> **Это важно проговорить: мой инструмент не заменяет sandbox, AV или полный reverse engineering. Он закрывает один технически сложный слой.**

7. Может ли нормальный файл получить findings?

Короткая позиция:

Да, поэтому проект использует severity, confidence, mitigations и manual review, а не делает категоричный вывод по одному признаку.

Подробный ответ:

Нормальные бинарники могут иметь нестандартную компоновку, защитный runtime, JIT-like механизмы, общий handler, unusual section names или toolchain-specific особенности. Поэтому многие признаки имеют medium severity, пороги, условия sample size и confidence penalties. Например, no_runtime_functions ослабляется, потому что leaf-only image может быть легитимен. moderate reuse требует corroboration. Strict ABI warnings не повышают suspicion score. Это показывает, что модель не пытается обвинять любой нестандартный файл.

Если комиссия давит:

Если спросят про ложные срабатывания: да, они возможны, поэтому результат называется finding, а не verdict of malware.

2. Предметная область PE, x64 и unwind

8. Почему проект анализирует только PE32+ AMD64, а не x86, ARM64, ELF или Mach-O?

Короткая позиция:

Потому что предмет ВКР - x64 Windows exception/unwind, а у других платформ другие форматы, правила и структуры.

Подробный ответ:

На x64 Windows stack unwinding основан на .pdata/RUNTIME_FUNCTION и UNWIND_INFO. В x86 Windows исторически используется другая модель SEH, стековые frames и compiler/runtime особенности. ARM64 имеет свои unwind-структуры. ELF/Mach-O вообще относятся к другим загрузчикам и форматам метаданных. Если смешать все платформы в одной ВКР, проект потеряет глубину и проверяемость. Ограничение PE32+ AMD64 - это не слабость, а способ сделать анализ точным и корректным.

Если комиссия давит:

> **Лучше глубоко и правильно покрыть один механизм, чем поверхностно заявить поддержку всех форматов.**

9. Что такое .pdata и почему она важна?

Короткая позиция:

.pdata содержит RUNTIME_FUNCTION entries - карту функций, нужную для поиска UNWIND_INFO по текущему RIP.

Подробный ответ:

На x64 Windows обработка исключений и раскрутка стека требуют знания, какая функция покрывает текущий адрес инструкции. RUNTIME_FUNCTION хранит BeginAddress, EndAddress и UnwindInfoAddress. По Begin/End определяется функция, по UnwindInfoAddress читается UNWIND_INFO. Если .pdata повреждена, перекрыта, несортирована или указывает за границы, virtual unwind может быть неверным. Поэтому значительная часть findings относится именно к function table integrity.

Если комиссия давит:

> **.pdata - это не второстепенная таблица. Для x64 unwinding это центральная карта выполнения.**

10. Что такое UNWIND_INFO и почему проект уделяет ему столько внимания?

Короткая позиция:

UNWIND_INFO описывает, как функция строит stack frame и как из него корректно выйти при exception/unwind.

Подробный ответ:

UNWIND_INFO содержит version, flags, размер prolog, count codes, unwind operations, frame register, stack allocation, saved registers, chained info и optional handler. Эти данные позволяют восстановить состояние caller frame без полного исполнения программы. Если UNWIND_INFO невозможно разобрать или он содержит странные opcode/operand/chain/handler поля, анализ стека может стать недостоверным. Для reverse engineer это критически важно: можно понять, где функция сохраняла регистры, где handler, и почему stack walking ломается.

Если комиссия давит:

> **UNWIND_INFO - это инструкция для раскрутки кадра. Если она повреждена, история выполнения может быть восстановлена неправильно.**

11. Почему нельзя просто дизассемблировать .text и не смотреть unwind metadata?

Короткая позиция:

Дизассемблирование и unwind metadata отвечают на разные вопросы и дополняют друг друга.

Подробный ответ:

Дизассемблер показывает инструкции, но unwind metadata является официальным описанием того, как Windows должна раскручивать стек и вызывать обработчики. В оптимизированном x64-коде prolog/epilog, tail calls, chained records и handlers могут быть сложными. Кроме того, задача проекта - не полный анализ всех инструкций, а проверка согласованности metadata. Полное сравнение prolog с UNWIND_INFO потребовало бы надежного disassembler и анализа control flow, что сильно расширило бы область ВКР. Проект сознательно остается в

стабильной зоне PE/unwind metadata.

Если комиссия давит:

> **Я не отрицаю пользу дизассемблера. Я анализирую другой слой - тот, которым пользуется механизм unwinding.**

12. Почему важно проверять границы файла и SizeOfImage?

Короткая позиция:

Потому что без корректных границ невозможно надежно перевести RVA в реальные байты и секции.

Подробный ответ:

Физический размер файла задает предел чтения raw bytes. SizeOfImage задает предел виртуального образа, внутри которого должны жить RVA-структуры. Если секция выходит за конец файла, она описывает несуществующие байты. Если section, RUNTIME_FUNCTION или UnwindInfoRVA выходит за SizeOfImage, адрес не принадлежит нормальному image mapping. Это не эвристическая вкусовщина, а базовая проверка координатной системы. Проект не придумывает эти границы, он проверяет согласованность с PE-файлом и моделью загрузчика Windows.

Если комиссия давит:

> **Если координатная система сломана, все последующие выводы могут быть ошибочными. Поэтому границы проверяются в начале.**

13. Кто устанавливает эти границы: вы или спецификация?

Короткая позиция:

Формальные границы задаются PE-структурами, файлом и Windows loader. Проект только проверяет согласованность с ними.

Подробный ответ:

Размер файла задается самим файлом на диске. SizeOfImage, SectionAlignment, FileAlignment, NumberOfSections и DataDirectory задаются PE headers, обычно сформированными линкером. Размер RUNTIME_FUNCTION entry и формат UNWIND_INFO задаются Windows x64 ABI. Мои собственные инженерные границы - это только процентные пороги интерпретации: 1%, 5%, 10%, 30%, 80%. Они не являются требованиями PE-формата, а нужны для разделения единичного артефакта и системного паттерна.

Если комиссия давит:

> **Формальные bounds не мои. Мои - только thresholds evidence model, и они явно задокументированы.**

14. Почему COFF Header не вошел в 27 findings?

Короткая позиция:

COFF Header проверяется как precondition, а не как suspicious finding.

Подробный ответ:

COFF Header отвечает за Machine, NumberOfSections, SizeOfOptionalHeader и общие характеристики файла. Если Machine не AMD64 или Optional Header некорректен, проект не может корректно применять x64 unwind-модель. Это не значит, что файл подозрительный по EH/unwind. Это значит, что он неподдерживаемый или невалидный вход для данного анализатора. Поэтому ошибки COFF/Optional Header должны быть parse diagnostics, а не scoring findings.

Если комиссия давит:

> **Неподдерживаемый формат - это причина остановить анализ, а не начислять подозрительность.**

3. Признаки, веса, пороги и confidence

15. Почему именно 27 признаков, а не 10 или 100?

Короткая позиция:

Потому что 27 - это минимально достаточная evidence matrix по слоям x64 EH/unwind, без ухода в общий malware analysis.

Подробный ответ:

27 признаков получаются из декомпозиции механизма: 6 PE/Exception Directory, 7 function table, 2 UNWIND_INFO integrity, 3 reuse/dominance, 5 placement, 4 combo/correlation. Если сделать меньше, исчезнут слепые зоны: например, не будет различия между rf_oob, invalid_ranges и rf_overlaps. Если сделать больше, начнется дробление одной проблемы на мелкие дубли и добавление общих PE-индикаторов вроде imports, entropy, timestamp. Поэтому 27 - это не случайное число, а компромисс между полнотой, объяснимостью и низким шумом.

Если комиссия давит:

> **Это не магическое число. Это результат послойной декомпозиции предметной области.**

16. Почему не добавлены entropy, imports, strings, certificate и другие признаки?

Короткая позиция:

Потому что они относятся к общему malware analysis, а не к EH/unwind metadata.

Подробный ответ:

Entropy может намекать на упаковку, imports - на поведение, strings - на контекст, certificate - на доверие к издателю. Но ни один из этих признаков не объясняет, корректна ли .pdata, читается ли UNWIND_INFO, куда указывает HandlerRVA, лежит ли metadata в executable/writable section. Включение таких признаков размыло бы тему ВКР и сделало бы проект похожим на неполный антивирус. Я сознательно ограничил evidence matrix только тем, что связано с x64 exception/unwind.

Если комиссия давит:

> **Я не делал универсальный malware classifier. Я делал объяснимый анализатор конкретной низкоуровневой области.**

17. Почему веса именно +25, +20, +15, +10, +8? Почему не +47 или +26?

Короткая позиция:

Потому что score - инженерная шкала уровней evidence, а не математически точная вероятность.

Подробный ответ:

Вес отражает относительную разрушительность проблемы для анализа. OOB, битая exception directory, overlap functions и массовый parse fail получают высокий вес, потому что ломают базовую возможность анализа. Placement и dominance получают средние веса, потому что имеют больше benign-объяснений. Combo получают добавочные веса, потому что усиливают уже найденные первичные сигналы, но не должны дублировать их полностью. +26 вместо +25 ничего принципиально не меняет, а усложняет объяснение. Поэтому веса округлены до понятных уровней evidence.

Если комиссия давит:

> **Я защищаю не микроточность одного балла, а прозрачную и стабильную ранжировку силы признаков.**

18. Почему порог 1% используется для некоторых ошибок?

Короткая позиция:

1% отделяет единичный дефект от повторяющегося повреждения в жестких integrity-признаках.

Подробный ответ:

Для invalid_ranges, rf_oob, unwind_rva_oob и unwind_parse_fail даже одна ошибка важна, но единичный дефект может быть артефактом, поврежденной записью или особенностью tiny sample. Если таких записей больше 1%, это уже выглядит не как случайность, а как системная проблема function/unwind table. Поэтому ниже или равно 1% применяется medium/low weight, а выше 1% - high weight. Confidence дополнительно учитывает small_sample_size, чтобы маленькие таблицы не давали переуверенности.

Если комиссия давит:

1% - это не закон PE, а инженерная граница между одиночным дефектом и системной аномалией.

19. Почему для unwind anomalies и unusual/writable placement используется 5%, а не 1%?

Короткая позиция:

Потому что эти признаки более вариативны и чаще имеют легитимные исключения.

Подробный ответ:

Format/OOB ошибки жестче: структура либо выходит за границу, либо нет. А вот placement и отдельные anomalies могут зависеть от toolchain, упаковщика, runtime, custom sections и build flags. Поэтому модель дает им более мягкий high threshold - 5%. Это снижает риск ложной тревоги от единичных нестандартных записей. Если же необычное размещение или anomalies затрагивают больше 5%, это уже широкий паттерн, который заслуживает более сильной оценки.

Если комиссия давит:

Разные пороги отражают разную природу признаков: жесткие format errors строже, placement patterns мягче.

20. Почему reuse moderate начинается с 10%, strong с 30%, а handler dominance с 80%?

Короткая позиция:

Потому что reuse и dominance являются статистическими pattern-признаками, а не прямыми ошибками формата.

Подробный ответ:

Reuse одного UNWIND_INFO может быть нормальным: много функций могут иметь одинаковый prolog. Поэтому moderate reuse требует больше 10%, n > 200 и дополнительный статический сигнал. Strong reuse требует больше 30%, что уже похоже на очень сильную концентрацию metadata. Handler dominance требует handler_count > 50 и reuse ratio > 80%, потому что единичные маленькие выборки не должны давать громкий finding. Порог 80% означает подавляющее большинство handler entries, а не просто частое повторение.

Если комиссия давит:

Эти thresholds сделаны консервативными, потому что статистический pattern без контекста легко переоценить.

21. Чем score отличается от confidence?

Короткая позиция:

Score показывает силу подозрительных signals, confidence показывает уверенность в интерпретации этих signals.

Подробный ответ:

Score складывается из весов findings и ограничен 0..100. Confidence строится отдельно: base, бонусы за семейства evidence, strong/medium findings, hard table/unwind issues и penalties за одиночность, маленькую выборку, placement-only или single-family evidence. Поэтому возможна ситуация: finding есть, score не ноль, но confidence снижен. Это правильно, потому что слабый одиночный сигнал не должен звучать как уверенный вывод.

Если комиссия давит:

Score отвечает 'насколько много и сильно', confidence - 'насколько надежно это интерпретировать'.

22. Почему confidence не является вероятностью?

Короткая позиция:

Потому что модель не обучалась как статистический классификатор на размеченном корпусе.

Подробный ответ:

Вероятность требует калиброванной статистической модели, dataset, ground truth, validation methodology и оценки ошибок. Здесь модель эвристическая и объяснимая. Confidence - это инженерная уверенность в интерпретации evidence, а не вероятность malware. Это честнее и научно аккуратнее, чем назвать число вероятностью без статистической основы. В документации прямо указано, что confidence is not a statistical probability.

Если комиссия давит:

Я не выдаю эвристику за ML-вероятность. Это сильная сторона, а не слабость: модель честно ограничена.

23. Зачем нужны combo-признаки, если первичные признаки уже есть?

Короткая позиция:

Combo показывают корреляцию сигналов между слоями и повышают убедительность связанной картины.

Подробный ответ:

Один placement finding может иметь benign-объяснение. Одна статистическая доминанта handler тоже может быть runtime-особенностью. Но когда необычное placement совпадает с integrity damage, или writable metadata совпадает с writable handler, это уже более сильный связанный pattern. Combo имеют небольшие добавочные веса +8 или +10, чтобы не дублировать первичные признаки, а только усилить интерпретацию их совместного появления.

Если комиссия давит:

> **Combo - это не повтор, а проверка того, что проблемы сходятся в одну evidence-картину.**

24. Почему strict ABI warnings не повышают suspicion score?

Короткая позиция:

Потому что совместимые production toolchains могут давать формальные ABI-отклонения, которые не являются подозрительностью сами по себе.

Подробный ответ:

Режим --strict-abi полезен для диагностики, но если автоматически повышать score за каждое формальное ABI-предупреждение, инструмент начнет шуметь на легитимных binaries. Поэтому strict ABI diagnostics остаются видимыми per function, но не меняют suspicion score, evidence level, verdict или confidence по умолчанию. Это консервативное решение: лучше показать предупреждение аналитику, чем превращать его в подозрительность без контекста.

Если комиссия давит:

> **Я разделяю диагностическую строгость и suspiciousness. Это снижает false positives.**

25. Почему score ограничен 100, если findings может быть много?

Короткая позиция:

Ограничение 0..100 делает score читаемой triage-шкалой и предотвращает бессмысленный рост.

Подробный ответ:

Если суммировать все веса без clamp, несколько связанных problems могли бы дать огромное число, которое трудно интерпретировать. Шкала 0..100 привычна и удобна: low, moderate, high suspicion. Важны не бесконечные баллы, а переходы между уровнями и сами findings. Поэтому даже при score 100 аналитик должен смотреть evidence families, affected counts и manual validation steps.

Если комиссия давит:

> **Score - навигация по отчету, не математическая мера всего риска.**

4. Архитектура и реализация

26. Почему проект написан на C, а не на Python?

Короткая позиция:

C подходит для низкоуровневого бинарного парсинга, контроля памяти, переносимости CLI и близости к PE/Windows API.

Подробный ответ:

Проект работает с байтовыми структурами, фиксированными offsets, little-endian чтением, Windows Debug API, PE headers и строгими границами. C дает точный контроль над layout, типами uint32_t/uint64_t, памятью и ошибками чтения. Такой инструмент проще поставлять как самостоятельный CLI без зависимости от Python runtime на машине аналитика. Python удобен для прототипа, тестов и вспомогательных скриптов, но core analyzer на C лучше показывает системное программирование и соответствует низкоуровневой теме ВКР.

Если комиссия давит:

> **Python был бы удобнее для скрипта, но C лучше раскрывает предмет: бинарные форматы, память, границы, Windows API и переносимый CLI.**

27. Почему не C++?

Короткая позиция:

C выбран как более минимальный и прозрачный язык для парсера, где важны явные структуры, ownership и отсутствие лишней абстракции.

Подробный ответ:

C++ дал бы RAII, STL и более богатые абстракции, но для этой ВКР это не обязательно. Проект не строит сложную объектную модель, а последовательно читает PE, таблицы, unwind structures и формирует отчеты. C делает каждую операцию с буфером и границей явной. Это полезно для защиты: можно показать, где проверяется bounds, где выделяется память, где формируется finding. При этом проект использует C17, CMake, sanitizers и тесты, чтобы компенсировать риски ручного управления памятью.

Если комиссия давит:

> **C++ не был бы ошибкой, но C здесь достаточно выразителен и лучше подчеркивает низкоуровневую природу задачи.**

28. Как вы защищаетесь от ошибок памяти в C?

Короткая позиция:

Проект использует safe read helpers, explicit bounds checks, sanitizers, unit tests, parser mutations и fuzzing entry point.

Подробный ответ:

Главный риск C - buffer overread, overflow и неправильное управление памятью. Поэтому чтение идет через функции, которые проверяют границы buffer. Сложение размеров делается безопасно, учитываются overflow cases, ranges проверяются через uint64_t там, где нужно. В README указаны sanitizer profiles: AddressSanitizer и UndefinedBehaviorSanitizer, coverage, regression corpus, deterministic parser mutations и libFuzzer campaign в CI. Это не делает C магически безопасным, но показывает системную работу с рисками.

Если комиссия давит:

> **Я не утверждаю, что C безопасен сам по себе. Я показываю меры: bounds checks, sanitizers, tests, fuzzing и ограниченный ownership.**

29. Как устроена архитектура проекта?

Короткая позиция:

Архитектура разделена на parser, unwind decoder, heuristics, report generation, dynamic collector, tests и Ghidra importer.

Подробный ответ:

В src/include находятся parser, heuristics, reporting и dynamic collector. PE parser отвечает за заголовки, секции, directories и преобразование координат. Unwind module читает RUNTIME_FUNCTION и UNWIND_INFO. Heuristics

module считает findings, score, confidence и mitigations. Report module формирует JSON, HTML, CSV и Ghidra CSV.Dynamic mode через Windows Debug API собирает runtime events. Такое разделение важно: finding logic несмешана с HTML, PE parsing не смешан с dynamic tracing, а отчеты отделены от core analysis.

Если комиссия давит:

Архитектура не монолитная: каждый слой имеет свою ответственность и проверяется отдельно.

30. Почему отчеты сразу в JSON, HTML, CSV и Ghidra CSV? Это не лишнее?

Короткая позиция:

Это разные потребности аналитика: машина, человек, таблица функций и импорт в reverse engineering workflow.

Подробный ответ:

JSON нужен для автоматической обработки, schema validation и CI. HTML нужен для удобного визуального отчета без внешних assets. CSV нужен для плоской таблицы функций и unwind fields. Ghidra CSV нужен, чтобы перенести ranges и comments в Ghidra и продолжить ручной reverse engineering. Это не украшение, а practical analyst workflow: инструмент не только находит signals, но и помогает дальше работать с ними.

Если комиссия давит:

Отчеты - часть полезности проекта. Finding без удобного формата проверки хуже защищается на практике.

31. Почему static analysis cross-platform, а dynamic только Windows x64?

Короткая позиция:

Статический PE parsing можно делать где угодно, а runtime exception/debug behavior зависит от Windows Debug API.

Подробный ответ:

PE-файл можно открыть и разобрать на Linux, macOS или Windows, потому что это просто bytes. Но динамическая часть запускает target, ловит exceptions, VEH/VCH/UEF, calls to RtlAddFunctionTable и handler hits. Это требует Windows semantics и Windows Debug API. На другой ОС такой runtime слой будет недоступен или будет эмуляцией, что выходит за рамки ВКР.

Если комиссия давит:

Это осознанное разделение: статический анализ переносимый, runtime validation привязана к платформе выполнения.

32. Почему dynamic mode вообще нужен, если есть статический анализ?

Короткая позиция:

Динамика нужна не для полной sandbox-замены, а для дополнительного evidence и correlation.

Подробный ответ:

Статика показывает структуру файла: что записано в .pdata, .xdata, handlers и секциях. Динамика показывает, происходят ли runtime события: exceptions, handler hits, VEH/VCH/UEF registrations, RtlAddFunctionTable calls. Если статический handler-related finding совпадает с runtime handler hit, evidence становится сильнее. Но dynamic mode bounded and observational: он не является полной трассировкой или malware sandbox. Это дополнительный слой проверки.

Если комиссия давит:

Динамика не заменяет статику, а помогает подтвердить или уточнить ее сигналы.

33. Не опасно ли запускать анализируемый файл в dynamic mode?

Короткая позиция:

Да, поэтому README прямо говорит использовать isolated VM для untrusted samples.

Подробный ответ:

Dynamic mode executes the analyzed file. Это потенциально опасно для вредоносных samples. Поэтому проект документирует ограничение и предлагает запускать динамическую проверку в изолированной виртуальной машине. Это честная инженерная позиция. Многие инструменты malware analysis имеют dynamic mode, но они требуют sandbox или VM. Статический режим остается безопаснее, потому что не исполняет target.

Если комиссия давит:

> **Я не скрываю риск. Dynamic mode - optional, и безопасность запуска вынесена в документацию.**

34. Почему не использовали готовую PE-библиотеку вместо собственного парсера?

Короткая позиция:

Собственный парсер дает контроль над проверками границ, диагностикой и учебной ценностью ВКР.

Подробный ответ:

Готовая библиотека могла бы ускорить разработку, но проекту важно не просто прочитать PE, а явно проверять OOB, overlaps, alignments, exception directory, section permissions и фиксировать reasons. Собственный ограниченный parser под PE32+ AMD64 позволяет показать комиссии, что я понимаю структуру файла и контролирую каждый переход от RVA к bytes. При этом parser не пытается покрыть все PE-экзотики, а работает в заданной области.

Если комиссия давит:

> **Для ВКР важно было реализовать и объяснить core logic, а не спрятать ее за библиотекой.**

5. Тестирование, проверка и качество

35. Как вы доказываете, что проект работает правильно?

Короткая позиция:

Через unit tests, PE regression samples, schema validation, semantic validation, parser mutations, sanitizer profiles, Windows dynamic smoke tests и CI.

Подробный ответ:

README описывает normal CTest run: unit tests, six PE regression samples, JSON Schema and semantic report validation, dynamic-report validator tests и 120 deterministic parser mutations. Также есть sanitizer и coverage presets, Windows builds MSVC/MinGW, dynamic smoke scenarios и bounded fuzz campaign. Это разные уровни проверки: unit tests ловят локальную логику, samples проверяют end-to-end, schema валидирует контракт JSON, semantic tests проверяют согласованность полей, mutations проверяют устойчивость parser к поврежденным данным.

Если комиссия давит:

> **Качество доказывается не одним запуском, а набором проверок на разных слоях.**

36. Как вы тестировали статические, динамические и смешанные случаи?

Короткая позиция:

Статика проверяется regression samples и mutated parsers; динамика - Windows smoke scenarios; смешанный режим - correlation и final assessment consistency.

Подробный ответ:

Статические tests должны покрывать корректные PE, malformed PE, битые .pdata/.xdata, OOB, overlaps, parse failures и report consistency. Динамические сценарии запускаются на Windows, где можно наблюдать exceptions, handler hits, VEH/VCH/UEF и runtime function table calls. Смешанный режим важен там, где статические placement/handler findings сопоставляются с runtime events. В отчете final assessment должен учитывать static evidence, runtime evidence, completeness и correlation rules.

Если комиссия давит:

> **Я разделяю три уровня: что видно в файле, что видно при запуске, и где эти слои подтверждают друг друга.**

37. Что если тестовые samples слишком маленькие и не доказывают реальную работоспособность?

Короткая позиция:

Маленькие samples нужны для воспроизводимости, а real-world compatibility runner предусмотрен отдельно.

Подробный ответ:

Синтетические и regression samples хороши тем, что точно известно, какая аномалия в них создана. Это позволяет воспроизводимо проверить конкретный finding. Но они не заменяют реальный corpus. Поэтому README упоминает optional real-world compatibility runner, который валидирует reports against JSON Schema and semantic invariants без выполнения samples. Для ВКР важна связка: детерминированные samples для точности и real corpus workflow для совместимости.

Если комиссия давит:

> **Идеальный dataset можно расширять, но текущая тестовая стратегия уже покрывает unit, regression, mutations, reports и dynamic smoke.**

38. Как проверялись ложные срабатывания?

Короткая позиция:

Через mitigations, conservative thresholds, strict ABI separation и compatibility runs.

Подробный ответ:

Полная оценка false positives требует большого размеченного корпуса, и проект честно не заявляет статистическую калибровку. Но в инженерной модели предусмотрены меры против переобвинения: small_sample_size, single_signal_only, placement_only_signal, single_family_only, no scoring for strict ABI-only

warnings, corroboration for moderate reuse, handler dominance filtering for runtime-like code sections. Это снижает рискгромких выводов по одиночным и спорным signals.

Если комиссия давит:

> **Я не скрываю, что это эвристика. Я показываю, какие меры встроены, чтобы она не была агрессивной.**

39. Можно ли доверять отчету, если динамическая сессия завершилась timeout или неполно?

Короткая позиция:

Нужно доверять осторожно: runtime evidence capped at weak, correlation не усиливается.

Подробный ответ:

Evidence matrix указывает: incomplete session сохраняет raw events, но runtime evidence level capped at weak. Incomplete sessions cannot establish static/dynamic correlation and do not increase final confidence. Это правильное поведение. Если target завис, timeout, instrumentation incomplete или no natural exit, проект не должен делать сильный dynamic verdict. Он показывает события, но не переоценивает их.

Если комиссия давит:

> **Неполная динамика не превращается в сильное доказательство. Это осознанная защита от ложной уверенности.**

40. Что именно проверяет JSON Schema?

Короткая позиция:

Она проверяет контракт отчета: структуру, поля, типы и стабильность машинно-читаемого формата.

Подробный ответ:

JSON отчет предназначен не только для человека, но и для автоматической обработки. Schema помогает гарантировать, что report содержит ожидаемые поля: metadata, findings, assessment, dynamic events, counters и т.д. Semantic validation идет дальше: проверяет cross-field consistency, например function sizes, counters and final-assessment consistency. Это важно для качества: отчет не должен быть красивым, но внутренне противоречивым.

Если комиссия давит:

> **Schema - это контракт API отчета, a semantic validation - проверка смысла между полями.**

41. Зачем нужны parser mutations и fuzzing?

Короткая позиция:

Потому что бинарный parser должен устойчиво обрабатывать поврежденные и специально испорченные inputs.

Подробный ответ:

PE-анализатор работает с недоверенными файлами. Любая ошибка bounds checking может стать crash или memory bug. Deterministic parser mutations проверяют известные классы повреждений: битые размеры, offsets, counts, flags. Fuzzing ищет неожиданные комбинации байтов. В сочетании с ASan/UBSan это помогает ловить overread, overflow, undefined behavior и некорректные assumptions.

Если комиссия давит:

> **Для парсера недоверенных бинарников fuzzing и sanitizers - не роскошь, а нормальная инженерная практика.**

6. Ограничения и честная защита слабых мест

42. Какие главные ограничения проекта?

Короткая позиция:

PE32+ AMD64 only, no x86/ARM64/ELF/Mach-O, static mode does not execute or disassemble .text, dynamic is Windows-only and bounded.

Подробный ответ:

Проект не поддерживает PE32/x86, ARM64, ELF, Mach-O и Intel APX UNWIND_INFO V3. Он не является полной sandbox, не строит complete runtime trace, не дизассемблирует весь .text и не доказывает malware. Fixed stack size derived from unwind operations and does not include runtime-sized allocations such as alloca. Эти ограничения не скрыты в коде, а явно указаны в README. Для ВКР это правильно: область применения должна быть четкой.

Если комиссия давит:

> **Ограничения не разрушают проект, если они честно названы и соответствуют цели работы.**

43. Если проект не дизассемблирует .text, как он может говорить о корректности prolog?

Короткая позиция:

Он не делает полного утверждения о соответствии prolog инструкциям, а проверяет metadata-level consistency.

Подробный ответ:

Проект анализирует UNWIND_INFO как структуру и проверяет version, flags, count codes, opcodes, operands, code offsets, chain, handler bounds, placement и table integrity. Он может сказать, что metadata не читается или внутренне странна. Но он не утверждает, что полностью сравнил каждую unwind operation с реальными инструкциями prolog. Это честное ограничение. Полное сравнение потребовало бы disassembler, CFG и архитектурного анализа инструкций.

Если комиссия давит:

> **Я не выдаю metadata checks за full binary verification. Это именно static metadata analyzer.**

44. Можно ли обойти ваш анализ, если специально сделать корректную metadata?

Короткая позиция:

Да. Если attacker оставит EH/unwind metadata корректной и не будет трогать этот слой, проект может не найти findings.

Подробный ответ:

Любой анализатор, ограниченный конкретной областью, можно обойти, если атакующий не использует эту область. Это не делает анализ бесполезным. Он полезен там, где упаковщики, protectors, obfuscators, manual function tables, fake handlers или runtime function tables оставляют следы в EH/unwind metadata. Правильная позиция: проект обнаруживает определенный класс структурных и поведенческих signals, а не гарантирует обнаружение всех угроз.

Если комиссия давит:

> **Обходимость не равна бесполезности. Это специализированный слой evidence, а не универсальный детектор.**

45. Что если файл использует нестандартный компилятор или packer и поэтому выглядит странно, но не вредоносен?

Короткая позиция:

Тогда findings становятся поводом для manual review, а не автоматическим обвинением.

Подробный ответ:

Проект специально разделяет suspicion, confidence и manual validation. Нестандартный compiler/protector может давать unusual sections, handler dominance, reuse patterns или writable sections. Поэтому findings не называются malware labels. Отчет показывает affected count, representative RVA, section, severity и manual steps. Аналитик проверяет, является ли это benign build behavior, packer, protector или malicious tampering.

Если комиссия давит:

Finding означает 'проверь это место', а не 'файл вредоносен'.

46. Почему dynamic mode не является полноценной sandbox?

Короткая позиция:

Потому что он собирает ограниченные события EH/unwind, а не весь behavior target.

Подробный ответ:

Sandbox обычно отслеживает filesystem, registry, network, processes, injection, API calls, persistence, anti-analysis и многое другое. Dynamic mode проекта фокусируется на exceptions, handler hits, VEH/VCH/UEF, RtlAddFunctionTable/RtlInstallFunctionTableCallback и bounded RIP trace. Это deliberately narrow runtime validation. Он нужен для correlation с EH/unwind static findings, а не для полного malware behavior report.

Если комиссия давит:

Я не строил sandbox. Я добавил динамический слой, релевантный теме ВКР.

47. Почему проект не использует машинное обучение?

Короткая позиция:

Потому что цель - explainable heuristic model, а не black-box classification.

Подробный ответ:

ML мог бы быть полезен при большом размеченном корпусе, но тогда главной проблемой стали бы dataset, labels, bias, explainability и calibration. Для ВКР по низкоуровневому PE/unwind важнее показать структуру, инварианты, thresholds и ручную проверяемость. Evidence matrix можно открыть, обсудить, изменить и протестировать. Это лучше подходит для защиты, чем модель, которая говорит 'так решил классификатор'.

Если комиссия давит:

Объяснимость здесь важнее потенциальной точности черного ящика.

7. Сравнение, практическая ценность и защита от провокаций

48. Почему вы считаете проект сложным по сравнению с сайтом или CRM?

Короткая позиция:

Сложность здесь не в UI, а в низкоуровневых бинарных структурах, границах, форматах и корректной интерпретации.

Подробный ответ:

Сайт или CRM часто сложны бизнес-логикой, интерфейсами, БД и UX. Этот проект сложен другим: нужно корректно читать недоверенный binary input, проверять overflow/OOB, понимать PE headers, section table, RVA-to-offset, .pdata, UNWIND_INFO, handler flags, Windows Debug API, JSON schema, tests и runtime correlation. Ошибка в одном offset может испортить весь отчет. Поэтому сложность инженерная и системная, а не интерфейсная.

Если комиссия давит:

> **Это системное ПО и binary analysis, где главное - корректность низкоуровневой интерпретации.**

49. Почему ваш проект не просто набор if-ов?

Короткая позиция:

Потому что if-ы реализуют формализованную evidence matrix, связанную с архитектурой x64 EH/unwind.

Подробный ответ:

Любая эвристическая система в коде в итоге содержит условия. Вопрос не в наличии if-ов, а в том, имеют ли они предметный смысл, документированы ли веса, есть ли семьи evidence, confidence, mitigations, tests и отчеты. В проекте conditions соответствуют PE/unwind инвариантам: bounds, overlaps, parse validity, section permissions, reuse ratios, handler dominance и correlations. Это не случайные проверки, а структурированная модель.

Если комиссия давит:

> **Да, технически это условия. Но они выражают предметные инварианты и проверяются тестами.**

50. Если антивирусы являются черным ящиком, почему ваш score тоже не черный ящик?

Короткая позиция:

Потому что все признаки, веса, пороги, mitigations и rationale явно документированы.

Подробный ответ:

Черный ящик скрывает, почему получен verdict. В моем проекте finding имеет code, family, severity, affected count, representative RVA, message. Evidence matrix показывает веса и thresholds. Confidence model показывает base, bonuses и penalties. Manual validation steps объясняют, что проверить в Ghidra/PE viewer/debugger. Поэтому пользователь может восстановить путь от bytes до verdict. Это принципиально отличается от 'модель сказала suspicious'.

Если комиссия давит:

> **Мой score не идеален, но он прозрачен. Его можно проверить и критиковать по конкретным правилам.**

51. Что в проекте можно было бы развивать дальше?

Короткая позиция:

ARM64/x86 support, deeper prolog disassembly comparison, larger real corpus calibration, richer dynamic tracing and more precise per-function correlation.

Подробный ответ:

Дальнейшее развитие логично идти по нескольким направлениям. Первое - поддержка других architectures и форматов unwind. Второе - интеграция disassembler для проверки соответствия UNWIND_INFO реальному prolog/epilog. Третье - расширение corpus и threshold calibration на реальных binaries. Четвертое - richer dynamic mode: больше event types, точнее correlation between static HandlerRVA and runtime hits. Пятое - UI или plugin для Ghidra/IDA. Эти направления не отменяют текущий результат, а показывают roadmap.

Если комиссия давит:

Я понимаю границы версии 1.0.0 и могу обоснованно сказать, куда проект масштабируется.

52. Какой самый сильный аргумент в защиту проекта?

Короткая позиция:

Проект решает узкую, технически сложную и объяснимую задачу end-to-end: от PE bytes до отчета и проверки.

Подробный ответ:

Сильная сторона не в том, что он 'ловит все вирусы'. Сильная сторона в полном инженерном цикле: парсинг недоверенного PE, decoding x64 UNWIND_INFO, evidence matrix, score/confidence, mitigations, schema reports, HTML/CSV/Ghidra integration, static/dynamic correlation, tests, sanitizers and release readiness. Это демонстрирует системное программирование, понимание Windows internals, defensive analysis и зрелое отношение к ограничениям.

Если комиссия давит:

Я защищаю не громкое обещание, а качественно реализованный специализированный анализатор.

53. Как отвечать, если комиссия говорит: 'это слишком узко'?

Короткая позиция:

Узость здесь является преимуществом, потому что позволяет глубоко и проверяемо решить конкретную задачу.

Подробный ответ:

Широкие проекты часто становятся поверхностными: немного imports, немного entropy, немного UI, немного ML. Мой проект выбирает одну сложную область и прорабатывает ее глубже: PE layout, .pdata, UNWIND_INFO, handlers, placement, dynamic correlation, reports and tests. В инженерии узкая специализация нормальна, особенно в reverse engineering. Вопрос не в широте, а в том, насколько полезно и корректно решена выбранная задача.

Если комиссия давит:

Это не 'маленькая тема', а глубокий слой Windows binary analysis.

54. Как отвечать, если комиссия говорит: 'это слишком сложно для пользователя'?

Короткая позиция:

Инструмент предназначен не для обычного пользователя, а для аналитика, которому нужна низкоуровневая детализация.

Подробный ответ:

Обычный пользователь не должен читать UNWIND_INFO. Но reverse engineer, malware analyst, преподаватель или студент системного направления как раз нуждаются в таких деталях. При этом проект снижает сложность: формирует HTML dashboard, JSON, CSV, Ghidra CSV, manual validation steps и объяснимые findings. То есть он не прячет сложность предмета, а структурирует ее.

Если комиссия давит:

Сложность интерфейса соответствует сложности предметной области, но отчет помогает ее разложить.

8. Короткие ответы на самые опасные вопросы

55. Ваш проект может ошибаться?

Короткая позиция:

Да, потому что это эвристический анализатор, поэтому он выдает evidence и manual review, а не абсолютный приговор.

Подробный ответ:

Ошибки возможны в двух направлениях: false positives и false negatives. Проект снижает false positives через thresholds, mitigations и confidence, а false negatives - через покрытие нескольких EH/unwind слоев. Но он не претендует на универсальное обнаружение. Научно честная позиция сильнее, чем обещание невозможной точности.

Если комиссия давит:

> **Да, может. Поэтому я называю результат finding, а не доказательство.**

56. Вы можете доказать, что ваши 27 признаков оптимальны?

Короткая позиция:

Нет, математической оптимальности я не заявляю. Я заявляю инженерно обоснованную, документированную и тестируемую матрицу.

Подробный ответ:

Оптимальность потребовала бы огромного labeled corpus, formal objective function и статистической оценки. Это другая работа. Моя цель - explicit, reviewable and stable heuristic logic. Признаки отобраны по предметной релевантности, покрытию слоев, ручной проверяемости и практической ценности.

Если комиссия давит:

> **Я защищаю обоснованность, а не математическую оптимальность.**

57. Что будет, если удалить один из признаков?

Короткая позиция:

Зависит от признака, но в целом появится слепая зона или потеря детализации отчета.

Подробный ответ:

Если удалить PE layout checks, можно анализировать неверные координаты. Если удалить function table checks, не будет контроля карты функций. Если удалить placement, не будет видно metadata в executable/writable sections. Если удалить combo, отчет станет плоским и потеряет correlation evidence. Поэтому признаки не равнозначны, но каждый закрывает свой участок.

Если комиссия давит:

> **Матрица построена по покрытиям слоев, поэтому удаление признака должно быть обосновано заменой, а не просто сокращением.**

58. Почему итоговый verdict не просто 'malicious/benign'?

Короткая позиция:

Потому что проект не имеет права делать такой вывод только по EH/unwind metadata.

Подробный ответ:

Malicious/benign - это глобальная классификация файла. EH/unwind findings - один слой evidence. Поэтому verdict использует high/moderate/low suspicion static/correlated, runtime_signals_only и case strength. Это точнее и честнее, чем бинарная метка, которая могла бы ввести пользователя в заблуждение.

Если комиссия давит:

> **Я не называю файл вредоносным без достаточной области evidence.**

59. Что если комиссия попросит объяснить проект за 30 секунд?

Короткая позиция:

Это инструмент для объяснимого анализа x64 PE exception/unwind metadata.

Подробный ответ:

Он читает PE32+ AMD64, проверяет section layout и exception directory, разбирает .pdata/RUNTIME_FUNCTION и UNWIND_INFO, находит 27 классов аномалий, считает прозрачный score/confidence, формирует JSON/HTML/CSV/Ghidra reports и при необходимости собирает Windows runtime evidence по exceptions и handlers. Его цель - помочь reverse engineer понять, где именно в EH/unwind слое файл выглядит поврежденным, необычным или требующим ручной проверки.

Если комиссия давит:

Короткая фраза: от PE bytes до объяснимого отчета по x64 unwind/EH.

Финальная защитная речь на 1 минуту

Мой проект не пытается заменить антивирус или sandbox. Он решает более узкую, но технически сложную задачу: объяснимый анализ x64 PE exception/unwind metadata. Инструмент читает PE32+ AMD64, проверяет корректность section layout и exception directory, разбирает RUNTIME_FUNCTION и UNWIND_INFO, выявляет структурные нарушения, подозрительное размещение metadata, reuse/dominance patterns и корреляции между слоями. Результат - не черный ящик: каждый finding имеет code, family, severity, affected count, representative RVA и объяснение. Score используется как triage aid, confidence отделен от suspiciousness, а спорные случаи ослабляются mitigations. Проект имеет статический и Windows dynamic режимы, отчеты JSON/HTML/CSV/Ghidra CSV и тестовую базу с unit, regression, schema, semantic validation, mutations, sanitizers, fuzzing и smoke scenarios. Его ценность - не в обещании обнаружить все угрозы, а в прозрачной диагностике сложного EH/unwind-слоя, который полезен reverse engineer, malware analyst и учебно-исследовательской защите.

Финальная фраза

Я защищаю не универсальный детектор вредоносности, а специализированный, объяснимый и проверяемый инструмент анализа x64 PE exception/unwind metadata.

Конец документа.